



MACO: A HW-Mapping Co-optimization Framework for DNN Accelerators

Wujie Zhong, Zijun Jiang and Yangdi Lyu*

Microelectronics Thrust, The Hong Kong University of Science and Technology (Guangzhou)

Guangzhou, Guangdong, China

{wzhong679, zjiang438}@connect.hkust-gz.edu.cn, yangdilyu@hkust-gz.edu.cn

ABSTRACT

Deep neural network (DNN) accelerators have been developed to enhance the effectiveness of DNN models, particularly in resource-constrained devices. Achieving high-throughput and energy-efficient inference within area constraints requires careful consideration of design choices in both hardware (HW) and mapping spaces. To get better performance and energy efficiency, it is important to optimize hardware and mapping together. However, this co-optimization process presents a considerable challenge due to the expansive combined HW-Mapping design space. To find the optimal configuration in this large design space, we formulate the exploration of the hardware configuration as an optimization problem, and embed the exploration of the mapping design space into the evaluation stage of optimization. We implement a HW-Mapping co-optimization framework called MACO to find optimal configurations for both hardware and mapping, and provide a generic interface to integrate different optimization algorithms, including multi-objective Bayesian optimization (MOBO), non-dominated sorting genetic algorithms (NSGA), and random search. We evaluate our framework with four popular DNN models of different properties. Our evaluation shows that the MOBO-based approach can achieve a 30% energy reduction and a 37% latency reduction with the same area as the state-of-the-art HW-Mapping optimization framework.

CCS CONCEPTS

• **Computer systems organization** → **Architectures**; • **Computing methodologies** → **Artificial intelligence**; **Machine learning**.

KEYWORDS

Deep Neural Network, Hardware Accelerator, Multi-Objective Bayesian Optimization

ACM Reference Format:

Wujie Zhong, Zijun Jiang and Yangdi Lyu. 2025. MACO: A HW-Mapping Co-optimization Framework for DNN Accelerators. In *30th Asia and South*

*Corresponding author

This work is supported by the Guangzhou Municipal Science and Technology Project (Municipal Key Laboratory Construction Project, Grant No. 2023A03J0013).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASPDAC '25, January 20–23, 2025, Tokyo, Japan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0635-6/25/01

<https://doi.org/10.1145/3658617.3697542>

Pacific Design Automation Conference (ASPDAC '25), January 20–23, 2025, Tokyo, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3658617.3697542>

1 INTRODUCTION

Deep Neural Networks (DNNs) [3, 16, 22] have become increasingly popular for a wide range of applications, such as image classification, speech recognition, and natural language processing. However, the growing complexity and scale of DNN architectures present significant computational challenges, particularly when deploying them on resource-constrained edge devices. To address this challenge, domain-specific accelerators have been developed to optimize the performance and energy efficiency of DNN models.

In the development of DNN accelerators, many configurations in both hardware (HW) and mapping space have a significant effect on the power, performance, and area (PPA) metrics [1, 6, 12, 13, 26]. Among the HW configurations, the capacity of data buffers, the number of PEs, and the architecture of network-on-chip (NoC) play a fundamental role in achieving an optimal balance between the PPA metrics. Once the hardware configuration is determined, we need to decide how to map the workload onto the hardware, which can be formulated as a factorization and permutation problem. The mapping process factorizes the dimensions of the workload and then chooses the appropriate loop order for each memory level. A good mapping can fully utilize the hardware resources to enhance throughput and energy efficiency.

Prior research [9, 10, 15, 24] has demonstrated the need to explore both the HW and mapping space to get better PPA metrics. Existing HW-mapping optimization frameworks [9, 15] co-optimize the hardware and mapping spaces at each layer of a DNN model, and then use heuristic algorithms like genetic algorithms (GA) to combine different configurations and determine the final configuration for the entire model. While this approach may be efficient at the layer level, the vast design space during the combination stage may hinder search efficiency and result in suboptimal design choices. For example, in a DNN model with 30 layers, each containing 100 Pareto optimal points, the combination space for the model is estimated to be on the order of 10^{60} . This exponential increase in complexity is attributed to the challenge of merging the Pareto set of each layer, which cannot be simply merged in linear time. Given the vast design space, combining optimal configurations layer-by-layer is a challenging task.

To address this challenge, we formulate the exploration of the hardware configuration as an optimization problem, and embed the exploration of the mapping design space into the evaluation stage of optimization. In contrast to the existing approaches, where the co-optimization problems are split based on DNN layers, we treat the whole DNN models as one workload, and co-optimize the

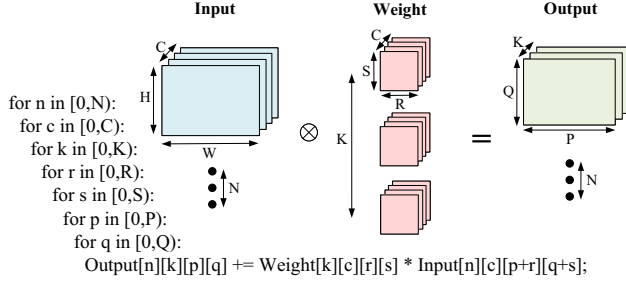


Figure 1: A convolution layer is represented as a 7D loop nest.

hardware and mapping space simultaneously. As there exist many excellent tools to generate optimal mappings for a given hardware configuration, such as CoSA [5] and LEMON [14], we will use them in the evaluation stage to get the optimal mapping, and evaluate the corresponding energy consumption and latency. Consequently, the HW-Mapping space exploration problem is transformed into a hardware space exploration problem, and can be solved using many optimization algorithms. We implement an open-source HW-Mapping co-optimization framework called MACO¹ to provide a generic interface to integrate many optimization algorithms, such as multi-objective Bayesian optimization (MOBO), non-dominated sorting genetic algorithms (NSGA), and random search for the hardware design search. Our primary contributions can be summarized as follows:

- We reduce the HW-Mapping space exploration problem into a hardware space exploration problem by integrating the mapping space exploration into the evaluation stage, and treat the whole evaluation stage as a blackbox function for the hardware space exploration block to optimize.
- We implement a HW-Mapping co-optimization framework called MACO to provide a generic interface to integrate different search algorithms, including MOBO, NSGA-II, and Random Search, for hardware space search, and various MIP models, LEMON [14] and CoSA [5], for mapping space search.
- We evaluate MACO on four popular DNN models. Experimental results demonstrate that our MOBO-based approach can achieve a 30% reduction in energy consumption and a 37% reduction in latency while maintaining the same area as the state-of-the-art HW-Mapping co-optimization baseline.

2 BACKGROUND AND RELATED WORKS

2.1 DNN and Accelerators

DNN layers can be represented as a series of nested loops. Fig. 1 demonstrates this representation using a seven-dimensional (7D) loop nest with specific loop boundaries for batch size (N), input channels (C), output channels (K), filter width (R), filter height (S), and spatial dimensions of the output feature map (P, Q). In this example, the convolution stride is 1. As a result, the input height H and weight W can be expressed as $P + R - 1$ and $Q + S - 1$.

¹<https://github.com/HKUSTGZ-MICS-LYU/MACO>

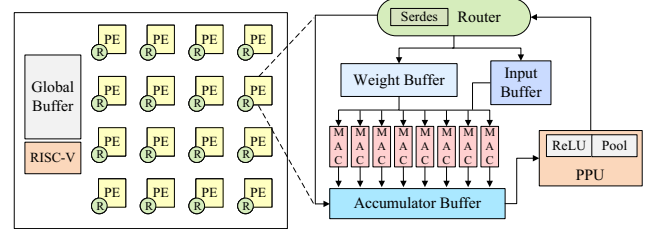


Figure 2: The architecture of a Simba chiplet.

DNN accelerators are specifically designed to enhance the performance of neural network computations, surpassing general-purpose architectures by utilizing the single instruction multiple data (SIMD) paradigm. Simba [18] is a specialized DNN accelerator tailored for convolutional neural networks (CNNs), which plays a vital role in computer vision applications. The detailed architecture of a Simba chiplet is depicted in Fig. 2, comprising a processing element (PE) array and a global buffer. Each PE includes local buffers, multiply-and-add (MAC) units, a router, and a post-process unit (PPU).

2.2 Hardware Space Exploration

To achieve the optimal PPA metrics, it is necessary to explore the hardware configurations for each DNN model. Taking the Simba chiplet as an example, there are a lot of configurations to be explored. At the chiplet level, designers need to decide the parameters including the size of the global buffer, the number of PEs, and the topology of the NoC. At the PE level, designers also need to consider the parameters including the sizes of local buffers and the number of MACs.

Due to the huge search space and time-consuming evaluation process, it is extremely challenging to explore the entire design space exhaustively. As a result, many previous studies have utilized various techniques to explore the hardware space. For example, Confucius [7] uses reinforcement learning (RL) and Interstellar [25] uses a two-step heuristic method for the hardware space exploration, both of which achieved better results compared to baseline designs.

2.3 Mapping Space Exploration

Once the hardware architecture is determined, we need to map the workloads onto the hardware design. As illustrated in Section 2.1, a convolutional layer can be represented as a seven-dimensional loop nest. The mapping process is to allocate the data of each dimension to each memory level.

Many previous studies have focused on mapping space exploration for a fixed hardware configuration. Timeloop [11] employs an exhaustive and random search method to find the optimal design. However, the large design space presents a significant challenge, making it impractical for exhaustive search to find the optimal mapping configuration in a limited time. To address the challenge, GAMMA [8] encodes the mapping space into a genome and employs a genetic algorithm to find the optimal solution. CoSA [5] formulates the mapping problem into equations and utilizes an MIP solver to find the solution, which yields superior efficiency than

Timeloop. Similarly, LEMON [14] formulates energy and latency into mathematical equations and also applies an MIP solver, which achieves superior outcomes than CoSA.

2.4 HW-Mapping Co-optimization

To achieve the optimal hardware and mapping configuration for a specific DNN model, it is crucial to co-optimize the HW-Mapping space. Many previous studies [9, 10, 15, 24] have utilized various methods to explore this space. DiGamma [9], which is built on Gamma, uses a genetic algorithm to encode the hardware and mapping space and outperforms GAMMA. However, its usage is limited to a basic two-level memory hardware architecture. DOSA [4] and the approach presented in [10] formulate the HW-Mapping space into equations and applies gradient-based methods to solve it. However, these approaches are designed to find a single solution, failing to identify the Pareto optimal set, which is necessary for balancing performance, power, and area (PPA) metrics.

MEDEA [15] integrates the HW-Mapping space and uses a multi-objective genetic algorithm called NSGA-II [2] to find the Pareto frontier corresponding to the PPA metrics for a Simba-like chiplet. The proposed approach in MEDEA consists of two steps: generating a Pareto optimal HW-Mapping configuration for each layer and then combining these configurations to create an overall configuration for the entire model. However, the combination space can be very large, especially for DNN models with multiple layers, leading to suboptimal solutions. Experimental evidence demonstrating this limitation can be found in Section 4.

3 MACO FRAMEWORK

In this section, we describe our problem formulation, and demonstrate our framework in detail.

3.1 High-level Overview

The goal of the MACO framework is to find optimal hardware configurations and mapping strategies that can minimize the objectives, including energy consumption, latency, and area. Given a fixed workload and design constraints, the objectives are a function of the hardware configuration (c) and the mapping strategy (m).

$$Obj = Eval(c, m) \quad (1)$$

To find the optimal configurations, we design our framework as Fig. 3, which consists of two main blocks: the hardware space search (HSS) block and the evaluation block.

1) **Hardware space search block.** The HSS block takes in the design, the constraint, and the search configurations as input, and generates the next hardware configuration for evaluation. The evaluation process is treated as a blackbox function, represented by Equation 2, and optimization algorithms are employed to approximate this function and find the optimal hardware configurations.

$$Obj(c) = \min_{m \in Mappings} Eval(c, m) \quad (2)$$

During the optimization process, the HSS block samples a set of hardware design points for assessment. For each sampled hardware architecture c^* , the evaluation block will be applied to calculate the objectives.

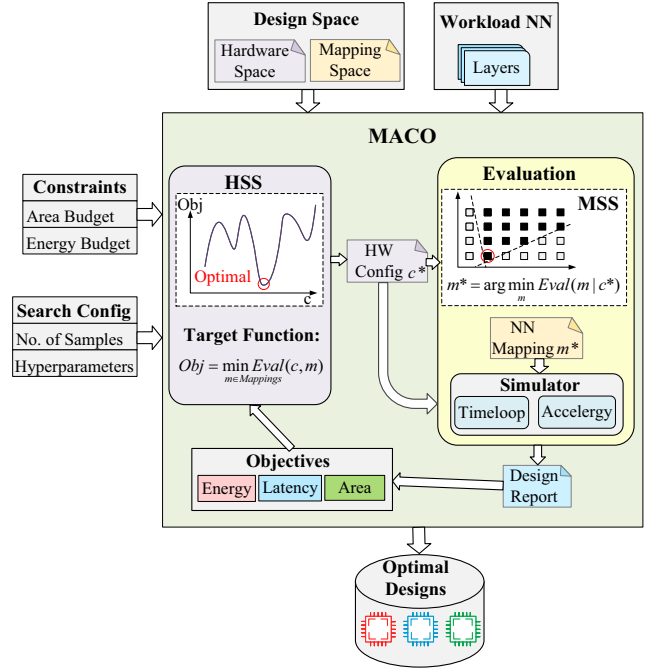


Figure 3: The overview of the MACO framework.

2) **Evaluation block.** To evaluate the objectives of a given hardware architecture c^* , a mapping space search (MSS) block is first invoked to determine the optimal mapping for the given workload and hardware architecture. Specifically, the optimal mapping m^* is determined by the following equation.

$$m^* = \arg \min_m Eval(m | c^*) \quad (3)$$

Once both the hardware architecture c^* and the mapping strategy m^* are determined, state-of-the-art DNN hardware simulation frameworks, such as Timeloop and Accelergy [23], are used to evaluate the objectives.

As we can see, the whole evaluation process is complex and cannot be expressed as a simple formula. The goal of HSS is to learn the complex evaluation equation through queries and predict optimal hardware configurations with minimal samples. Furthermore, this framework allows the HSS and MSS components to independently select algorithms without conflict. In our framework, the HSS block integrates different algorithms such as NSGA-II, MOBO, and Random Search, and the MSS part integrates various MIP models, such as CoSA [5] and LEMON [14]. All future improvements in each of them can enhance the overall framework.

3.2 Hardware Design Space

The HSS block is designed to analyze the memory and computation units of the Simba-like accelerator to determine the optimal configuration for a given workload. To define the range of each component within the design space, we use the format *begin:end:step*, with the size of the space indicated by *space size*, as shown in Table 1.

Table 1: The hardware design space.

Components	Depth	Number	Space Size
Global Buffer	512:3584:128	1	24
Input Buffer	128:1920:64	16	28
Weight Buffer	32:992:32	128	30
Acc Buffer	32:224:16	128	12
MAC	-	512:2048:128	12

Table 1 illustrates an example of the hardware design space, which includes four memory units and one compute unit. The exploration of the design space involves varying the buffer depth of different memory levels, from the global buffer to the PE accumulator buffer. In the example, the exploration range of the PE accumulator buffer is from 32 to 224, with an incremental step size of 16, which results in a design space size of 12 for the component. Regarding the compute unit, the exploration involves varying the MAC numbers from 512 to 2048, with a step size of 128. The total design space size is calculated to be 2.9 million, which is the product of the design space size of each component in the table.

The hardware design choice is denoted as the variable c , consisting of five dimensions, each corresponding to an encoded subspace. During each iteration of the HSS process, c will be decoded to a hardware configuration, and used as the input for the MSS block. Once c reaches its upper bound in all dimensions, the generated hardware will occupy the largest area among different design choices. The maximum allowable area will be set below the maximum design space area to allow for trade-offs between the computational and memory units. For instance, if one wants to increase MAC numbers, buffer sizes must be reduced to fit the area limitation.

3.3 HSS Algorithms

Algorithm 1 Pseudo-code for the MOBO Algorithm

Input: hardware design space C , the number of iterations N
Output: optimal configuration and objective pairs (C^*, O^*)

- 1: Sample initial dataset D from the design space C .
- 2: **for** i **from** 0 **to** N **do**
- 3: ▷ Exploration Stage
- 4: Fit the Gaussian Process model GP using dataset D .
- 5: Update the posterior distribution $p(o|c, D)$ on GP .
- 6: Sample dataset (Cs, Os) using the acquisition function $Acq(p(o|c, D))$.
- 7: Get the Pareto optimal set (Cp, Op) from (Cs, Os) .
- 8: Randomly select a value c^* from Cp as the next hardware design choice.
- 9: ▷ Evaluation Stage
- 10: Get the optimal mappings m^* from the MSS block for c^* .
- 11: Calculate the objectives o^* using the evaluation tools with the hardware configuration c^* and the mapping strategy m^* .
- 12: Update dataset D : $D \leftarrow D \cup (c^*, o^*)$.
- 13: **end for**
- 14: Calculate the Pareto optimal set (C^*, O^*) from D .
- 15: Return (C^*, O^*) .

After defining the hardware design space, the HSS block starts by sampling various design points and evaluating the objectives. In

each iteration, the HSS block generates a hardware configuration and utilizes the MSS block to determine the optimal mappings. The generated hardware and mappings are then evaluated to obtain the objectives. Due to the complex relationship between hardware design choice and objectives, which is hard to express in math equations, it is reasonable to employ heuristic methods to find solutions within limited sample points. We incorporate a range of algorithms in the MACO framework, including random search, NSGA-II, and MOBO, to identify the Pareto optimal set with minimal objectives. Our experiments in Section 4.2 demonstrate that MOBO yields the best results.

The MOBO algorithm is illustrated in Algorithm 1, which requires two inputs: the encoded hardware design space C and the number of evaluations N . Upon completion of its execution, the algorithm will generate the Pareto optimal set, denoted by (C^*, O^*) . The algorithm begins by sampling a predefined dataset from C and storing them in the initial dataset D (line 1). Then, it enters a loop that continues until the iteration number reaches N (line 2). In each iteration, it first enters the exploration stage, in which our algorithm tries to fit the evaluation function and predict the next point for evaluation. In our MOBO algorithm, the sampled dataset D is fit into a Gaussian Process model GP to derive the posterior distribution $p(o|c, D)$ (line 4-5), from which a set of points is sampled (Cs, Os) using the acquisition function Acq (line 6). The Pareto optimal points (Cp, Op) are then collected from this sampled dataset (line 7). Next, it randomly selects an input c^* from the Pareto set Cp as the next hardware design choice (line 8). In the evaluation stage, the MSS block first takes c^* as input to find the optimal mapping m^* for the DNN model (line 10). Then, the framework calculates the objectives o^* using the encoded design space c^* and the optimal mapping m^* (line 11). Finally, the selected point (c^*, o^*) is added to the dataset D , marking the end of the current iteration (line 12). Subsequently, a new iteration will be initiated. After completing all iterations, the final Pareto optimal set (C^*, O^*) is generated using the sampled dataset D (line 14). This set will be employed to get the corresponding hardware and mapping configurations.

The selection of the acquisition function in the MOBO algorithm is a critical factor that determines the subsequent design choice to be evaluated and significantly impacts the final outcomes. In this work, the upper confidence bound (UCB) [17] is utilized as the acquisition function, defined as follows.

$$Acq(p(o|c, D)) = \mu(p(o|c, D)) + \lambda\sigma(p(o|c, D)) \quad (4)$$

where $p(o|c, D)$ is the posterior distribution derived from the Gaussian Process model GP using the sampled dataset D , $\mu(p(o|c, D))$ is the mean value of the posterior distribution and $\sigma(p(o|c, D))$ is the corresponding standard deviation. The variable λ is a hyperparameter, which influences the potential reward of the unexplored regions in the GP model.

3.4 Evaluation Block

The evaluation block comprises the mapping space search (MSS) block and the DNN hardware simulators. The MSS block takes in the hardware configuration, the constraint, and the workload files to produce optimal mappings. It integrates various MIP models, such as CoSA [5] and LEMON [14], to efficiently and accurately explore the mapping space. Previous study [14] indicates that LEMON

outperforms CoSA in producing high-quality mappings. As a result, we utilize LEMON as the primary model for mapping space exploration in the majority of our experiments. This model takes hardware configuration and workload as input and transforms the mapping problem into mathematical equations that can be effectively solved using standard MIP solvers. Upon solving, LEMON generates the optimal mapping configuration, which is then evaluated by Timeloop and Accelergy to generate the objectives.

4 EXPERIMENT

4.1 Setup

4.1.1 DNN Models and Platform. The evaluation metrics of HW-Mapping configurations are power, performance, and area (PPA). In particular, we employ end-to-end energy consumption and latency (delay) to represent power and performance metrics. Our workloads involve four well-known CNN models (ResNet50 [3], VGG16 [19], EfficientNetB0 [21], and InceptionV3 [20]) with various characteristics.

We employ a variety of searching techniques to explore the HW-Mapping design space and establish uniform criteria for fair comparison. Firstly, we set an area limitation of 2.74 mm^2 , which is also the area constraint used in MEDEA [15]. As a result, only those hardware configurations within the area constraint are accepted. Secondly, we utilize Timeloop and Accelergy for the evaluation of all the techniques, both of which are widely used simulation frameworks in DNN accelerator modeling. Lastly, all the experiments are conducted on a 12th Generation Intel i7-12700 processor with 15 threads. These setups ensure that the environment remains standardized across all the experiments for comparative analysis of the search techniques.

4.1.2 Target Search Methods and Parameters. A previous study shows that LEMON [14] outperforms CoSA [5] in solving the mapping problem. Therefore, we integrate different heuristic methods with LEMON to co-optimize hardware and mapping. These enhanced methods are then compared against the state-of-the-art baselines. The detailed information about the state-of-the-art baselines and LEMON-based methods are as follows.

State-of-the-art baselines. We use MEDEA and LEMON as the baselines. MEDEA is a state-of-the-art HW-Mapping co-design framework, which can optimize the PPA metrics and generate the Pareto optimal solutions. On the contrary, LEMON focuses on the mapping space within a predefined hardware configuration. To ensure optimal performance from LEMON, we utilize the maximum hardware configuration in [15], which precisely conforms to the predefined area limitation.

LEMON-based methods. We integrate three algorithms with LEMON: Random+LEMON, NSGA-II+LEMON, and MOBO+LEMON. Each uses LEMON for mapping space but differs in hardware space exploration methods. Random+LEMON uses a stochastic search, NSGA-II+LEMON uses NSGA-II, and MOBO+LEMON uses a MOBO algorithm. All aim to find the Pareto front for PPA metrics. Each method samples 60 hardware points, taking 2-7 hours across different DNN models on our platform.

4.2 Pareto Front and Hypervolume

In the experiment, we employed different search methods to explore the HW-Mapping space and optimize the PPA metrics. The energy-delay product (EDP) and area trade-offs are shown in Fig. 4. The Pareto frontiers illustrated that the configurations found by our framework (Random + LEMON, NSGA-II + LEMON, and MOBO + LEMON) achieved better trade-offs compared to state-of-the-art approaches such as MEDEA and LEMON.

4.2.1 MEDEA. The EDP values of MEDEA are significantly larger than other methods. It implies the configurations found by MEDEA requires greater energy consumption and latency. This outcome can be attributed to the integration of the mapping and hardware space in MEDEA, resulting in a vast mapping space for any DNN model with multiple layers. As a result, MEDEA has limited capabilities in exploring the extensive design space, leading to suboptimal solutions.

4.2.2 LEMON. The data point for LEMON in Fig. 4 is denoted by an orange marker in all subplots, indicating a fixed hardware configuration with an area of 2.74 mm^2 , consistent with the predefined area limitation. LEMON formulates the mapping problem as equations and uses MIP to solve it. The EDP values of LEMON, as depicted in the figure, are significantly lower than those of MEDEA, demonstrating its ability to explore the mapping space.

4.2.3 The methods from our framework. The Pareto optimal sets found by the three LEMON-based methods are obviously better than those of LEMON, as shown in Fig. 4. The improvement is mainly attributed to their ability to explore the design space with a flexible trade-off between the compute and memory units.

Among the LEMON-based methods, MOBO+LEMON performs best. The Pareto optimal points of MOBO+LEMON exhibit dominance in the entire solution set of MEDEA in ResNet50 and InceptionV3. In VGG16 and EfficientNetB0, a subset of the MEDEA solutions are also dominated by those of MOBO+LEMON. Additionally, the entire Pareto optimal points found by Random+LEMON and NSGA-II+LEMON in ResNet50, VGG16, and EfficientNetB0 are dominated by those of MOBO+LEMON. However, deciding the superior optimization method for InceptionV3 remains challenging as the Pareto optimal points found by the three LEMON-based methods are less distinct. NSGA-II+LEMON is the second-best solution after MOBO+LEMON, demonstrating superior capability in dominating the solutions of Random+LEMON in most of the DNN models. Despite its less optimal performance, Random+LEMON still outperforms the original LEMON method.

Fig. 5 illustrates the normalized hypervolume for LEMON-based methods, with MOBO+LEMON demonstrating an average $2.84\times$ speedup compared to Random+LEMON and a $1.3\times$ speedup compared to NSGA-II+LEMON. These results indicate a substantially improved Pareto front obtained by MOBO+LEMON compared to the other two LEMON-based methods. The Bayesian model proves to be more effective in modeling the relationship between hardware parameters and multiple objectives in most DNN models, especially when dealing with limited data samples. Consequently, MOBO+LEMON outperforms NSGA-II in these models. However, MOBO+LEMON and NSGA-II+LEMON have similar hypervolumes when applied to the InceptionV3 model. It is worth noting that

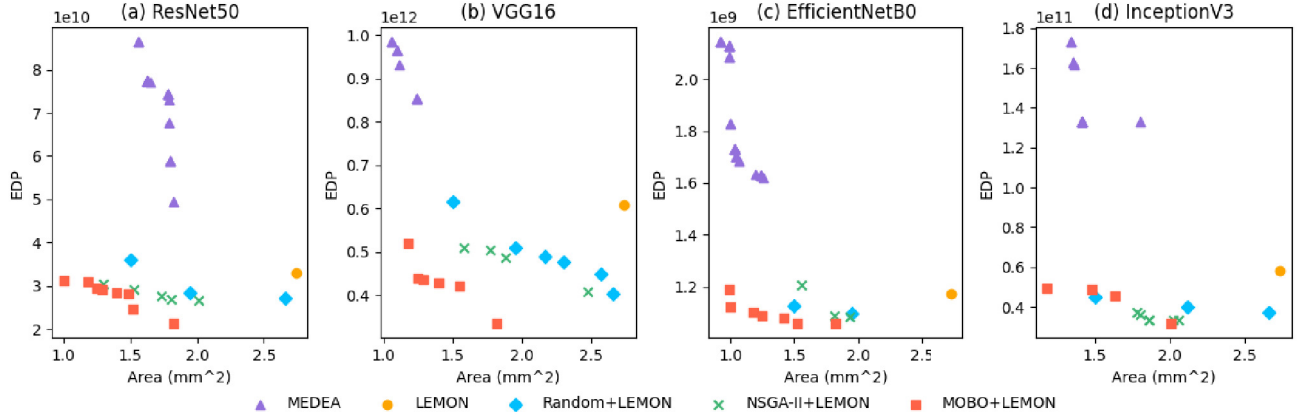


Figure 4: The Pareto optimal sets found by different search methods on (a) ResNet50 (b) VGG16 (c) EfficientNetB0 and (d) InceptionV3.

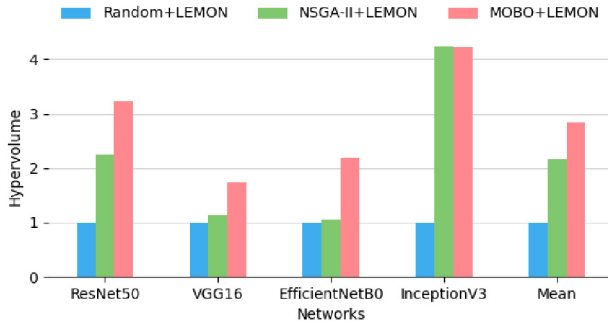


Figure 5: The normalized hypervolume of the LEMON-based methods

LEMON struggles to solve the mapping problems for certain hardware configurations within the time limit for the InceptionV3 model. Consequently, it is difficult to figure out the relationship between the parameters and objectives, leading to suboptimal solutions.

4.3 Energy and Latency Reduction

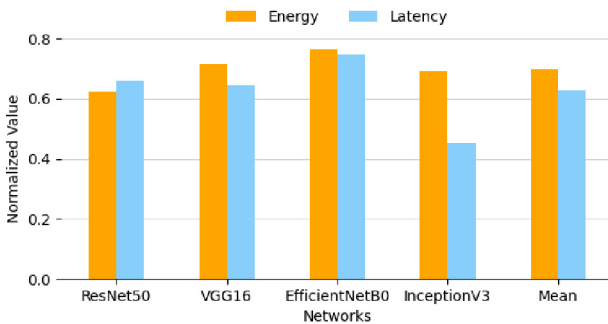


Figure 6: The normalized energy and latency values of MOBO+LEMON with respect to MEDEA. To make a fair comparison, We maintain the same area for the two methods.

The comparison of normalized energy consumption and latency values between MOBO+LEMON and MEDEA is illustrated in Fig. 6.

These values are obtained by calculating the ratio of data from MOBO+LEMON to that of MEDEA, where the data is collected from Pareto optimal points with the same area between the two methods to ensure a fair comparison. The results indicate that MOBO+LEMON yields a substantial reduction in both energy consumption and latency across different DNN models. On average, MOBO+LEMON achieves a 30% reduction in energy consumption and a 37% reduction in latency compared to MEDEA, while maintaining the same area. This is mainly because MOBO+LEMON can search the HW-Mapping design space more efficiently. Additionally, MOBO+LEMON offers a broader design space which is not limited by the area constraint, to enable trade-offs between compute and memory units. This approach successfully mitigates the exponential growth problem in mapping design space encountered by MEDEA when determining the final configuration from different DNN layers.

5 CONCLUSION

Co-optimizing HW and mapping is crucial for DNN accelerators to optimize PPA metrics. However, it demands a substantial effort to discover the optimal design choices because of the extensive design space. To address this challenge, we propose a HW-Mapping co-optimization framework called MACO. By leveraging advanced search algorithms such as MOBO, NSGA, and MIP, MACO effectively reduces the search space, enabling a more efficient and targeted exploration. Moreover, MACO provides a generic interface for easy integration of other algorithms. We evaluated MACO using four popular DNN models with different features. The experimental results show that MACO can achieve a 30% reduction in energy and a 37% reduction in latency while maintaining the same area as the state-of-the-art HW-Mapping optimization framework.

REFERENCES

- [1] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.
- [2] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.

- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [4] Charles Hoang, Qijing Huang, Grace Dinh, Mahesh Subedar, and Yakun Sophia Shao. 2023. Dosa: Differentiable model-based one-loop search for dnn accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 209–224.
- [5] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, James Demmel, John Wawrzynek, and Yakun Sophia Shao. 2021. Cosa: Scheduling by constrained optimization for spatial accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 554–566.
- [6] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [7] Sheng-Chun Kao, Geonhwa Jeong, and Tushar Krishna. 2020. Confucius: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 622–636.
- [8] Sheng-Chun Kao and Tushar Krishna. 2020. Gamma: Automating the hw mapping of dnn models on accelerators via genetic algorithm. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.
- [9] Sheng-Chun Kao, Michael Pellauer, Angshuman Parashar, and Tushar Krishna. 2022. DiGamma: Domain-aware genetic algorithm for HW-mapping co-optimization for DNN accelerators. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 232–237.
- [10] Rongjian Liang, Jianfeng Song, Yuan Bo, and Jiang Hu. 2022. Deep Learning Toolkit-Accelerated Analytical Co-optimization of CNN Hardware and Dataflow. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [11] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucec Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 304–315.
- [12] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Pugliesi, Rangharajan Venkatesan, Brucec Khailany, Joel Emer, Stephen W Keckler, and William J Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. *ACM SIGARCH computer architecture news* 45, 2 (2017), 27–40.
- [13] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 58–70.
- [14] Enrico Russo, Maurizio Palesi, Giuseppe Ascia, Davide Patti, Salvatore Monteleone, and Vincenzo Catania. 2023. Memory-Aware DNN Algorithm-Hardware Mapping via Integer Linear Programming. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*. 134–143.
- [15] Enrico Russo, Maurizio Palesi, Salvatore Monteleone, Davide Patti, Giuseppe Ascia, and Vincenzo Catania. 2022. MEDEA: A multi-objective evolutionary approach to DNN hardware mapping. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 226–231.
- [16] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4510–4520.
- [17] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. 2015. Taking the human out of the loop: A review of Bayesian optimization. *Proc. IEEE* 104, 1 (2015), 148–175.
- [18] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, et al. 2019. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 14–27.
- [19] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [20] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2818–2826.
- [21] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. PMLR, 6105–6114.
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [23] Yannan Nellie Wu, Joel S Emer, and Vivienne Sze. 2019. Accelergy: An architecture-level energy estimation methodology for accelerator designs. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [24] Qingcheng Xiao, Size Zheng, Bingzhe Wu, Pengcheng Xu, Xuehai Qian, and Yun Liang. 2021. Hasco: Towards agile hardware and software co-design for tensor computation. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1055–1068.
- [25] Xuan Yang, Mingyu Gao, Qiaoyi Liu, Jeff Setter, Jing Pu, Ankita Nayak, Steven Bell, Kaidi Cao, Heonjae Ha, Priyanka Raina, et al. 2020. Interstellar: Using halide's scheduling language to analyze dnn accelerators. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 369–383.
- [26] Zhekai Zhang, Hanrui Wang, Song Han, and William J Dally. 2020. Sparch: Efficient architecture for sparse matrix multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 261–274.